João Henrique Schmidt de Carvalho

# Unlocking Entity Component Systems in Game Engines: A Practical Approach to Core Mechanics

[CIDADE]

2026

João Henrique Schmidt de Carvalho

# Unlocking Entity Component Systems in Game Engines: A Practical Approach to Core Mechanics

Trabalho de Conclusão de Curso apresentado ao [CURSO] da [UNIVERSIDADE], como requisito parcial à obtenção do título de [GRAU].

[UNIVERSIDADE]

Supervisor: Geraldo Xexeo

[CIDADE]

2026

# RESUMO

Este trabalho explora a integração do padrão Entity Component System (ECS) e de scripting no desenvolvimento de motores de jogos, com foco em seu papel na melhoria de desempenho, manutenibilidade, modularidade e complexidade de código. O ECS oferece uma abordagem orientada a dados para gerenciar entidades e comportamentos de jogos, tratando desafios relacionados à localidade de memória, ao custo do polimorfismo e ao paralelismo. O scripting possibilita a interação dinâmica e estática com o motor, por meio de Lua e C++, respectivamente, oferecendo maior acessibilidade quando o ECS é desnecessário. Este estudo analisa o impacto do ECS em funcionalidades centrais, como renderização gráfica e simulação física. As principais contribuições incluem benchmarks de desempenho, análises arquiteturais e diretrizes práticas para integrar ECS e scripting a motores de jogos, com implicações que se estendem a softwares de simulação e a outras aplicações críticas em desempenho.

**Palavras-chave**: Entity Component System. Motores de jogos. Scripting. Desempenho.

# ABSTRACT

This work explores the integration of Entity Component System (ECS) and scripting into game engine development, with a focus on their roles in enhancing performance, maintainability, modularity and coding complexity. ECS provides a data-driven approach to managing game entities and behaviors, addressing challenges related to memory locality, polymorphism overhead, and parallelism. Scripting facilitates both dynamic and static interaction with the engine, through Lua and C++ respectivelly, allowing greater accessibility when the ECS is unnecessary. This study analyzes ECS impact on core functionalities like graphics rendering and physics simulation. Key contributions include performance benchmarks, architectural insights, and practical guidelines for integrating ECS and scripting into game engines, with implications extending to simulation software and other performance-critical applications.

**Keywords**: Entity Component System. Game engines. Scripting. Performance.

# LIST OF FIGURES

Figure 1 – Example of vTable storing the linkage between two derived classes . . . . . 15
Figure 2 – Movement System performs a scalar multiplication and a sum in two vectors 21
Figure 3 – Arbitrary components from arbitrary entities filled with arbitrary data in a random order . . . . . 26
Figure 4 – Big Entities, time per frame reading one component of each entity . . . . . 38
Figure 5 – Big Entities, time per frame reading all components of each entity . . . . . 38
Figure 6 – Memory Size, time per frame reading one struct of each entity . . . . . 39
Figure 7 – Memory Size, time per frame reading all structs of each entity . . . . . 39
Figure 8 – Particle Simulation, collision detection time per frame . . . . . 40
Figure 9 – Particle Simulation, kinematics time per frame . . . . . 40
Figure 10 – Particle Simulation, Render2d time per frame . . . . . 41

# LIST OF TABLES

Table 1 – walls, trees and power poles . . . . . 31
Table 2 – cars . . . . . 32

# CONTENTS


1 INTRODUCTION SCHEME . . . . . 7

2 IMPORTANT THEORETICAL FRAMEWORKS . . . . . 9

3 DESCRIPTION OF THE PROBLEM . . . . . 11
3.1 Description of the problem: Object Centric Loops . . . . . 11
3.2 Description of the problem: 3 problems of OOP flexibility . . . . . 14

4 ECS AS REMEDY . . . . . 17
4.1 ECS as remedy: System . . . . . 17
4.2 ECS as remedy: Components . . . . . 21
4.3 ECS as remedy: Entity . . . . . 25
4.4 ECS as remedy: Registry . . . . . 27
4.5 ECS as remedy: Archetypes Registries . . . . . 27
4.6 Explaining the ECS method: Scripting . . . . . 34

5 RESULTS . . . . . 37
5.1 Tests . . . . . 37
5.2 Big Entities Scene . . . . . 38
5.3 Memory Size Test . . . . . 38
5.4 Particle Simulation . . . . . 39
5.5 Threats to validity . . . . . 40
5.6 Conclusion . . . . . 41

References . . . . . 42

# 1 INTRODUCTION SCHEME

The creation of game engines is a complex task that involves integrating multiple systems, including graphics, physics, input handling, sound, events, camera, just to name a few. One of the main architectural design choice of most modern game engines is the presence of ECS, that isolates each of those tasks into “Systems”. Those have access to all data from the game.

ECS is a data-driven architecture, which forbades object methods and instead strips its data into engine components whenever possible. Alongside ECS, scripting plays a role when those components can’t be separated, and also enable dynamic interaction with interpreted languages, such as Python, Lua, among others. Both ECS and scripting allow for complex and simple game design choices respectivelly, which is essential for handling modern games.

This TCC focuses on how these systems are built and their relationship with other engine components, mainly graphics and physics. The goal is to design and implement these systems within a game engine, providing a comparasion with OOP of their impact on maintainability and then performance. Additionally, this work will explore how ECS and scripting can potentially influence other core systems, demonstrating their importance in the overall architecture of game engines

The significance of this research lies in the growing need for modular and efficient game engines. ECS has become a widely adopted pattern in the industry due to its ability to handle large-scale projects by decoupling data from logic, id est, the entity-component from systems.

Scripting, meanwhile, allows game developers to still use OOP when necessary and, allow to quickly change gameplay without touching the engine code; With added bonus if the scripting languages is interpreted since it won’t need to compile. In turn, also makes it easier to apply modifications by non-core developers. Understanding how scripting interact with the ECS Api can provide valuable insights for improving engine performance because they can decide when to pass script code into the engine code.

Meanwhile, even when not using such architecture, it can be helpful to know why. For example, Godot programers decided that, the extra maintainability created more disadvantages than the extra perfomance was worth it (Linietsky, 2021). Therefore, it is a decision that must be made knowing its trade-offs

First, I will talk about references and other essential tools used when building the ECS for a game engine or other big c-based projects in general.

Then I will research and define the core architecture of ECS, identifying how entities, components, and systems will interact within the engine. This includes designing the data structures and memory layout.

Follwoing that, I will briefly talk about two main sub-architectures

Next, I will show an implementation of ECS, the Registry and the Scene, the latter will act as a sort of API to manipulate complex data structures demonstrating.

Finally, I will conduct performance benchmarks and tests to evaluate the efficiency and scalability of the ECS in handling big scenes, showing some examples of its interaction with other engine systems.

By the end of this project, I expect to have contributed a functional ECS and scripting system within Pain-Engine, which is a game engine I'm building as a hobby, documenting how these systems either improve or challenge existing architectures developers job. This TCC will serve as a practical guide for developers looking to integrate ECS and scripting into their own projects, offering insights into the design choices, trade-offs, and challenges associated with these core desicions.

# 2 IMPORTANT THEORETICAL FRAMEWORKS

Computer Systems: A Programmer's Perspective by Randal Bryant is sometimes called "The Computer Science Book" by some of my colleagues. This reference is key for understanding the lower-level assembly aspects of coding, which are critical when building high-performance and clean software. It covers topics like memory management, CPU architecture, instructions, unexpected behaviors and efficient code execution, all of which are relevant to optimizing queries inside ECS's systems, and their data structures (Bryant; O'Hallaron, 2011).

The detailed exploration of how instructions are decided by the compiler will help me make informed decisions when managing resources in the engine, ensuring that it remains both efficient and scalable. This book was be especially helpful in addressing challenges related to safety problems, as it provides insights into what problems.

Another aspect of this book is that it covers some unintuitive and invisible behaviors from the c programming language that can lead to bugs, even for advanced programmers. By doing so, it justifies some of the design choices that I'm using in my game engine.

The EnTT source code by skypjack. EnTT is a well-established and highly efficient C++ library for implementing a Sparse-Set Entity Component System (ECS). I chose to reference EnTT because it provides a already exhaustively tested ECS architecture that has been widely adopted in both hobbyist and professional game developments, like Minecraft, Crimson Rush, ArcGIS Runtime SDKs, etc (Caini, 2024). I also choose because of his wonderful blog called "ECS back and forth" which goes in detail on what problems engines are usually trying to solve (Caini, 2019). Studying EnTT's source code will expose some intersting ways on how to create functions to handle entities and components.

Its use of modern C++ techniques, like template meta-programming, will serve as a valuable guide for implementing similar strategies in my own Registry system. The library is also made with the intent of being very simple to use and to modify if necessary.

ECST is an experimental multithreaded compile-time ECS library. It was developed as a Computer Science graduation project. I chose to reference ECST for its interesting approach to compile-time and multithreaded designs. The library was made using c++14, but by studying ECST's source code, one could gain knowledge to design an ECS that leverages modern c++20 architectures (Romeo, 2024).

Lastly and perhaps the closest to the present article is Sander Mertens Flecs ECS library, and also exhaustively tested on numerous projects like Tempest Rising, Territory Control 2, Resistance is Brutal, etc (Mertens, *[S.d.]*). Which is also complemented by the amazing blog

series “Building and ECS”. There, he disclosures what designs and principles were choosen during his making of Flecs (Mertens, 2022). Especially the main “Archetype” architecture, which differs from EnTT Sparse-Set.

# 3 DESCRIPTION OF THE PROBLEM

## 3.1 DESCRIPTION OF THE PROBLEM: OBJECT CENTRIC LOOPS

There is no way around this subject other than to describe exactly what is being built here. As with any work, its purpose is to solve problems, or at least to solve it in a marginally better way than other existing solutions. To be more precise, there are 2 problems an ECS intends to solve for game engines:

On the first problem, suppose that a developer is creating his/her game, and it needs to update millions of objects. Some of them are quite different from each other, so you have different structs of data.

For example, it might have objects like lights, meshes, audios, sprites, transforms, etc.

```
CLASS Mesh EXTENDS Node
CLASS Light EXTENDS Node
CLASS Audio EXTENDS Node
CLASS Sprite EXTENDS Node
CLASS Transform EXTENDS Node
```

Most probably, the object will be a combination of those above:

```
CLASS MeshWithAudio EXTENDS Node
  Mesh mesh
  Autio audio

CLASS LightWithAudio EXTENDS Node
  Light mesh
  Autio audio

CLASS LightMeshWithAudio EXTENDS Node
  Light light
  Mesh mesh
  Autio audio

CLASS CollisionWithMesh EXTENDS Node
  Collider collider
  Mesh mesh
// etc
```

Or, if you language support multiple inheritance, you can do the following:

```
CLASS MeshWithAudio EXTENDS Mesh, Audio
CLASS LightWithAudio EXTENDS Light, Audio
CLASS LightMeshWithAudio EXTENDS Light, Mesh, Audio
CLASS CollisionWithSprite EXTENDS Collider, Sprite
// etc...
```

Either way, using those primary objects, you would need to make it available for the developer, so it can be used on actual objects, e.g.:

```
CLASS Enemy EXTENDS ColliderSpriteWithAudio
CLASS Player EXTENDS ColliderSpriteWithAudio
CLASS Firefly EXTENDS LightWithAudio
CLASS Lamp EXTENDS LightMeshAudioCollider
CLASS Wall EXTENDS CollisionWithSprite
```

Therefore, you would need to define $2^N$ different classes, which is not feasable for higher values of $N$. Let's explore how Godot, which is an exclusively OOP engine, solve this problem inside their definiton of `Node` at `scene/main/node.h` (Godot Engine Contributors, *[S.d.]*)

```
class Node : public Object {
  struct Data {
    Node *parent = nullptr;
    Node *owner = nullptr;
    HashMap<StringName, Node *> children;
  };
  Data data;
};
```

Now, the problem is solved by using composition and trees, without using multiple inheritance. From Godot perspective, you can place new components inside its children hashmap, and as long as it is inherited from Node, you will have your object as complex as necessary.

```
Node enemy = new Node();
enemy.data.children.insert({"Mesh", new Mesh{}}, {"Audio", new Audio});
```

Now, if we were to add an interpreted language, we can expose a function such as `insert_component` to add new components to objects inside the engine, without needing to recompile.

With that ready, the developer also needs to iterate between all those objects and update their data. Each object will have at least some sort of action in the game, for example, be rendered, collide, process events, move, change states, etc.

Those direct approaches are prone to create two specific code styles to fill those tasks: a

vertical style that is object-centric (OC) and a more horizontal style that is system-centric (SC).

In the vertical style, each object is called and sequentially updated to perform specific tasks:

```
FOR EACH enemy IN Enemies
  enemy.move()
  enemy.checkCollisions()
  enemy.render()
  enemy.searchPlayer()
  // etc

FOR EACH player IN Players
  player.move()
  player.checkCollisions()
  player.render()
  // etc

FOR EACH lamp IN Lamps
  lamp.checkCollisions()
  lamp.render()
  // etc

FOR EACH firefly IN Fireflies
  firefly.move()
  firefly.render()
  // etc
```

A little problem emerges: in this example, enemies executes first meaning it can move and collide with the player first. But, in another simulation, if player executes first, then it can run before getting hit.

Notice however, that many of those functions are performing the same task for many different objects. So in OOP, we can solve the problem by using interfaces to our advantage:

```
INTERFACE i_move IMPLEMENTS move
INTERFACE i_render IMPLEMENTS render
INTERFACE i_collidable IMPLEMENTS checkCollisions

CLASS Enemy IMPLEMENTS i_move, i_collidable, i_render
CLASS Player IMPLEMENTS i_move, i_collidable, i_render
CLASS Lamp IMPLEMENTS i_collidable, i_render
CLASS Firefly IMPLEMENTS i_move, i_render

// ...later
movables.add(NEW Enemy(), NEW Player(), NEW Firefly())
renderables.add(NEW Enemy(), NEW Player(), NEW Lamp(), NEW Firefly())
collidable.add(NEW Enemy(), NEW Player(), NEW Lamp())
```

```
FOR EACH movable IN movables
  (i_move) movable.move()
FOR EACH renderable IN renderables
  (i_render) renderable.render()
FOR EACH collider IN collidable
  (i_collidable) collider.checkCollisions()
```

Why is this necessary? Mainly because it can happen that one operation conceptually needs to happen for all objects before another operation begins. Now enemies need to make their moves before checking their collisions, solving that little problem.

This is a SC loop but not an ECS yet. This specific style, which is common in OOP, still has 3 performance issues.

## 3.2 DESCRIPTION OF THE PROBLEM: 3 PROBLEMS OF OOP FLEXIBILITY

The first one is polymorphism overhead. Dynamic polymorphism in typical OOP implementations commonly uses an indirect dispatch mechanism. Why? Because whenever you call a method of a base class or using interfaces, the call itself involves one indirect step before the actual code is reached, and this redirection creates the overhead. Its cost usually depends on implementation, $C++$ usually does this with the use of vTables, which are very straightforward indexed tables that links virtual functions to real functions. But what is the overhead? At runtime the program needs to check inside the vTable what is the correct link between the virtual function and the actual function before performing the call. That extra step is what makes the program slow down (Driesen; Hölzle, 1996)

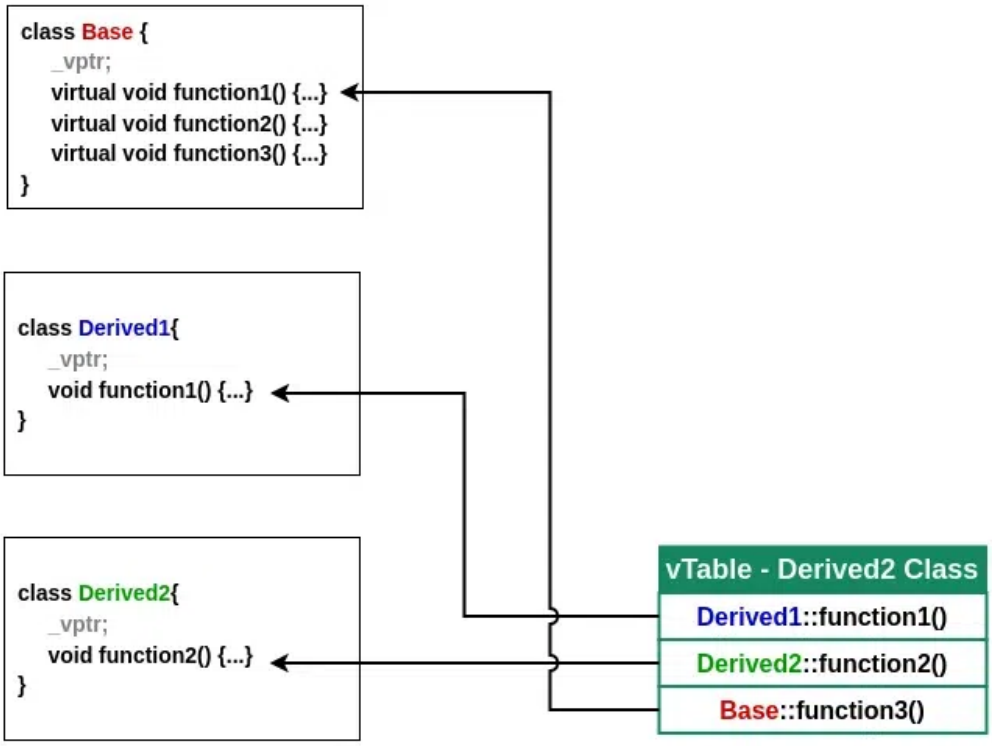
It's designed this way on purpose. For example, both enemies and fireflies implement a move function, but firefly function may not necessarly be the same as enemies since flies usually move on 3 axis, including up and down. Interfaces therefore, also allow you to modify the function bodies per class at the cost of a little memory and cpu.

The second problem is the memory locality. In the ECS case, the data will always be stored into a data structure of your choice, like an array or a map. Since you have the option to store all data in a single pool, you might as well store on a contiguous section in your memory.

However, when iterating in OOP, you are looping in the data of the entire object first before going to the next object. If such object has different members that arent't being used in the function, your instruction will have to skip them.

It is possible to clearly see the cost by doing a simple test. E.g, writing a small loop to sum all elements of a matrix "A":

Figure 1 – Example of vTable storing the linkage between two derived classes




Source: [SOURCE TO BE CONFIRMED]

```
result = 0
A = Matrix(20000,20000)

FOR EACH i from 0 to A.rows - 1:
  FOR EACH j FROM 0 TO A.cols - 1:
    ADD A[i][j] TO result
```

When implemented in C, this code will execute in 1.78 seconds for a square matrix A of size 20000

```
result = 0
A = Matrix(20000,20000)

FOR EACH j FROM 0 TO A.cols - 1:
  FOR EACH i from 0 to A.rows - 1:
    ADD A[i][j] TO result
```

When implemented in C, this code will execute in 2.23 seconds for a square matrix A of size 20000, a 25% increase

What is happening here? In the memory, the data for matrix A is stored in the following layout:

```
A[0][0], A[0][1], A[0][2], ..., A[0][N], A[1][0], ..., A[1][N], ..., A[N][N]
```

When the CPU reads memory, it generally does not fetch just the 4-byte float like we requested. It fetches a cache line, typically something like 64 bytes on modern CPUs. So, when requesting:

```
A[0][0], A[0][1], A[0][2], A[0][3], A[0][4], A[0][5], A[0][6], A[0][7], A[0][8], ...
```

The CPU can request the next number from his own cache instead of fecthing again.

If we choosen to not do it, we are essentially jumping 20000 bytes of integers for every single iteration, and this increase the cache misses we see on perf command. In the end, this just shows what is already known, it is better for the CPU if the loops operates with arrays that are contiguous in memory.

With this we conclude the two disadvantages of not using an ECS style to deal with objects in a simulation.

However, there is a third, very indirect problem: which is the difficulty of implementing parallelism when compared to ECS... To be more precise, the work needed to parallelize ECS systems is smaller than OOP cases because of the restrictions it impose.

I will talk more about this when discussing systems, but to make a brief introduction as to why: the main problem with parallelization is finding units of processing which don't share any data, but because data in ECSs often iterated is mostly with the same set of components, there is a guarantee that the data inside a single system is unrelated to each other, which allows easy concurrency.

# 4 ECS AS REMEDY

## 4.1 ECS AS REMEDY: SYSTEM

The previous OOP SC loop also allows for further flexibility when it comes to completing engine tasks. For example:

```
CLASS Firefly IMPLEMENTS i_move, i_render
  FUNCTION move OVERRIDE
    // move firefly ...
  // ... rest of the class

CLASS Enemy IMPLEMENTS i_move, i_collidable, i_render
  FUNCTION move OVERRIDE
    // move enemy ...
  // ... rest of the class
```

This is a useful flexibility to have, but it's also probable that the developer will implement some repeated calculation. For example, let's suppose a very simple firefly would use 5 members:

```
CLASS Firefly IMPLEMENTS i_move, i_render
  position = (0,0)
  velocity = (0,0)
  intensity = 1
  timer = 0.0

  FUNCTION move(dt) OVERRIDE
    timer -= dt

    // Choose random dir
    IF directionTimer <= 0.0f THEN
      velocity.x = getRandomNum()
      velocity.y = getRandomNum()
      timer = getRandomNum()

    // Apply velocity !
    position += velocity * dt

  FUNCTION render OVERRIDE
    // draw light
  // ... rest of the class
```

Similarly, a very simple enemy that just follows the player would also need to apply both

position and velocity, alongside other members:

```
CLASS Enemy IMPLEMENTS i_move, i_collidable, i_render
  position = (0,0)
  velocity = (0,0)
  collider = (1,1)
  speed = 1
  sprite = { TYPE::SQUARE, (1,1) }
  material = { "Texture.glsl", 0xFFFFFF }

  FUNCTION move(float dt) OVERRIDE
    direction = (0,0)

    // player dir
    direction = pathToPlayer()
    velocity = direction * speed;

    // Apply velocity !
    position += velocity * dt;

  FUNCTION checkCollisions OVERRIDE
    // collide with other colliders
  FUNCTION render OVERRIDE
    // draw sprite and material
  // ... rest of the class
```

The existance of a SC loop is already known by us. Since they are both using the same Euler formula, the formula itself can be moved to the SC loop:

```
FUNCTION applyVelocity(positions, velocities, dt)
  FOR EACH (position, velocity) IN ZIP(positions, velocities)
    position += velocity * dt
```

Doing so means the engine is running the behavior separately from the data, and this is correct. Systems are just code blocks that transforms data, each of them are functions that represent a specific logic, like applying velocity.

However a new problem emerges: when we try to create a system like `applyVelocity`, the underlying data isn't yet organized into convinient variables like `positions` and `velocities`. They are, instead, own by each object as member variables.

The solution is to just to separate them, removing the ownership of their respective objects and instead putting in a contiguous block of memory. For example, suppose we have one firefly and one enemy:

```
velocities = [(0,0), (0,0)]
positions = [(0,0), (0,0)]
```

The system now has everything to work, meaning it can now be called and therefore, perform its tasks. However, we quickly need to compensate for removing the ownership on both objects:

```
CLASS Firefly IMPLEMENTS i_move
  row = 0
  intensity = 1
  timer = 0.0

  PUBLIC FUNCTION move(dt) OVERRIDE
    velocity = velocities[row] // now retrieving from outside ownership
    timer -= dt

    // Choose random dir
    IF directionTimer <= 0.0f THEN
      velocity.x = getRandomNum()
      velocity.y = getRandomNum()
      timer = getRandomNum()
    // No need to apply velocity to position

  // ... rest of the class
CLASS Enemy IMPLEMENTS i_move, i_collidable, i_render
  row = 1
  collider = (1,1)
  speed = 1
  sprite = { TYPE::SQUARE, (1,1) }
  material = { "Texture.glsl", 0xFFFFFF }

  PUBLIC FUNCTION move(float dt) OVERRIDE
    velocity = velocities[row] // now retrieving from outside ownership
    direction = (0,0)

    // player dir
    direction = pathToPlayer()
    velocity = direction * speed;

    // No need to apply velocity to position

  // ... rest of the class
```

For small scale simulations where the number of objects is small, this setup can be slower since we are also calling a third extra function just to run two caclulations that were going to be done anyway. However, we are not interested in small simulations because those tend to already be performant anyway.

By only making this change, we already achieve a very crude ECS. Specific components of those objects (position, velocity) are separated, but they are also contiguous inside memory. Meaning they make better use the CPU cache. As the number of objects increase, the advantages in performance are sure to follow.

The downside is that we are not allowed to override a system function, since its just a static function. So if we need to make some task that is too complicated, we will inevitably add more if branches to it. An example of this can be seen inside the render system that will be shown later.

Notice, however, it sill has OOP elements on it, which is the polymorphism on methods like `move`, `render`, `checkCollision`, etc. That is completly fine, since ECS also allow those elements by including a scripting system.

With all this, let's see the ECS design for the engine tasks by creating different Systems:

```
// interface functions
FOR EACH movable IN movables
  (i_move) movable.move()
FOR EACH renderable IN renderables
  (i_render) renderable.render()
FOR EACH collider IN collidable
  (i_collidable) collider.checkCollisions()

// movement system (already seen)
FOR EACH movementComponent, transformComponent IN movements, transforms
  // move

// collisions system
FOR EACH collisionComponent, movementComponent, transformComponent IN colliders,
    movements, transforms
  // check collisions

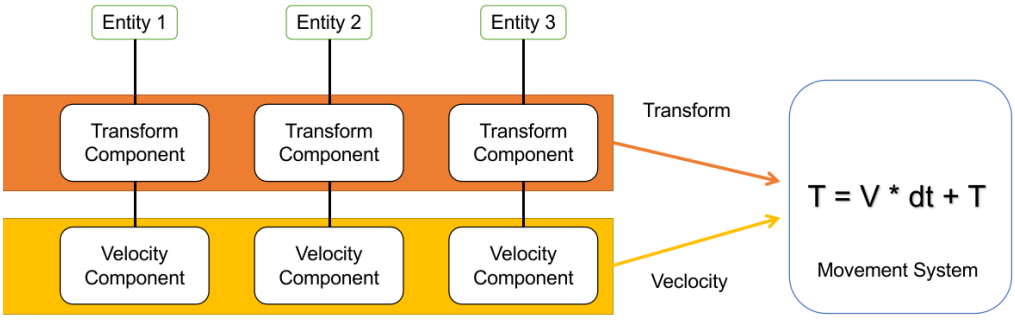
// render system
FOR EACH materialComponent, spriteComponent IN materials, sprites
  // render

// etc...
```

Using that, we solve memory locality, since the components are now close together in memory, but we still have the polymorphism overhead, because each system function still proceed an interface function. One way to solve this is to create more and more SC functions, until the entire object update is performed by systems.

Even if that isn't possible, since they will all use contiguous memory, they are at minimum still performing faster even if we include interfaces.

Figure 2 – Movement System performs a scalar multiplication and a sum in two vectors




Source: [SOURCE TO BE CONFIRMED]

But how to creaate more systems? As a rule of thumb, the code inside each loop is generally very specific, because their purpose is to update a very broad range of objects with a well-defined task. For example:

- A Movement/Kinetic System updates positions based on velocities
- A Collision System calculates the intersection between collision boxes
- A Render System handles filling the batch rendering buffer
- Lifetime System decrements timers and removes entities when their lifetime expires
- Particle System updates particle positions, velocities, colors, and lifetimes of swarms
- UI System updates UI elements based on application or gameplay state.
- Event System dispatches and processes gameplay functions between objects
- AI System updates decision-making, behavior trees, paths, or state machines
- Scripting System executes scripts attached to objects for everything else
- *etc*

The process of creating them follows the same principle of an optimization fluxogram: if any function proves to be source of constant bottlenecks, then moving the loop from OC to SC becomes more attractive.

## 4.2 ECS AS REMEDY: COMPONENTS

When expanding an object, we can expand its methods by polymorphism. However, the same isn't true for its data definition, which is always entirely copied to the derived class. Meaning that, the changes made to firefly and enemy examples are just different models of basic ownership: our model entirely move members into dedicated data structures, and this decision completly orthogonal to OOP.

Previously there were only 2 objects both with 1 set o components:

```
velocities = [(0,0), (0,0)]
positions = [(0,0), (0,0)]
```

However, our example was too shallow, it didn't show what to do when clases with different components are at play.

To make this concrete, let's go back to our modified classes that were left over from the OOP hierarchy: `Enemy` and a `Firefly`. On a whim, the developer also included a `Wall`, our third object.

We didn't touched the other members until now and our objective is to move the maximum amount of members into a contiguous memory, as well as removing functions that are already on the system. So let's do that.

```
CLASS Enemy IMPLEMENTS i_move
  row = 1
  speed = 1
  sprite = {}

  PUBLIC FUNCTION move OVERRIDE
    // still needs speed for the velocity module

  // ... rest of the class

CLASS Firefly IMPLEMENTS i_move
  row = 0
  timer = 0.0

  PUBLIC FUNCTION move(dt) OVERRIDE
    // still needs timer

  // ... rest of the class

CLASS Wall
  row = 2


velocities = [(0,0), (0,0), NULL]
positions = [(0,0), (0,0), (0,0)]
collider = [NULL, (1,1), (1,1)]
material = [NULL, {"Texture.glsl", 0xFFFFFF }, {"Texture.glsl", 0xAA11FF}]
sprite = [NULL, {TYPE::SQUARE, (1,1)}, {TYPE::RECT, (1,1)}]
light = [{intensity = 1}, NULL, NULL] // firefly light component was transported
```

We are starting to create a matrix of components. If 100 of each were initialized, they would generate the following matrix:

| Component | $enemy_0$ | $\cdots$ | $enemy_{99}$ | $firefly_0$ | $\cdots$ | $firefly_{99}$ | $wall_0$ | $\cdots$ | $wall_{99}$ |
|---|---|---|---|---|---|---|---|---|---|
| **TransformComponent** | $(0,0)$ | $\cdots$ | $(0,0)$ | $(0,0)$ | $\cdots$ | $(0,0)$ | $(0,0)$ | $\cdots$ | $(0,0)$ |
| **ColliderComponent** | $(1,1)$ | $\cdots$ | $(1,1)$ | — | $\cdots$ | — | $(1,1)$ | $\cdots$ | $(1,1)$ |
| **VelocityComponent** | $(0,0)$ | $\cdots$ | $(0,0)$ | $(0,0)$ | $\cdots$ | $(0,0)$ | — | $\cdots$ | — |
| **MaterialComponent** | `{"Texture.glsl", 0xFFFFFF}` | $\cdots$ | `{"Texture.glsl", 0xFFFFFF}` | — | $\cdots$ | — | `{"Texture.glsl", 0xAA11FF}` | $\cdots$ | `{"Texture.glsl", 0xAA11FF}` |
| **SpriteComponent** | `{TYPE::SQUARE, (1,1)}` | $\cdots$ | `{TYPE::SQUARE, (1,1)}` | — | $\cdots$ | — | `{TYPE::RECT, (1,1)}` | $\cdots$ | `{TYPE::RECT, (1,1)}` |
| **LightComponent** | — | $\cdots$ | — | `{intensity = 1}` | $\cdots$ | `{intensity = 1}` | — | $\cdots$ | — |

There are two main problems. The first is that, representing components as a matrix indexed by entity obligates each component array to reserve space for every entity, even when most entities do not possess that component. This creates sparse data structures: iterating over a component requires either checking for missing components or traversing unused entries, while memory is also consumed by slots that contain no component at all.

“How to solve” this problem is the question for the next 3 chapters and the core of ECS development and discussions. But first, the natural first step is to allow systems to select sets of components.

Why? Because Systems interact with the engine by asking for specific queries. Each should ask the ECS library what component it want to iterate with. Infering this logic, it is expected that each system has a yes/no decision for every component, meaning it is possible to use bitmask logic with it, something that CPUs happens to handle very well.

To do this, we can associate every component with a base 2 ID:

```
CLASS TransformComponent   // { (0,0), 0.0, (1,1) } position, rotation, scale
  STATIC id = 1
CLASS VelocityComponent   // { (0,0) } linear velocity
  STATIC id = 2
CLASS SpriteComponent     // { (0,1), (32,32) } texture rect, size
  STATIC id = 4
CLASS MeshComponent       // { "cube", "brick" } mesh handle, material
  STATIC id = 8
CLASS LightComponent      // { (1,0.9,0.7), 1.0, true } color, intensity, enabled
  STATIC id = 16
CLASS ColliderComponent   // { (16,16), false } half extents, isTrigger
  STATIC id = 32
CLASS AudioComponent      // { "step.wav", 1.0 } clip handle, volume
  STATIC id = 64
CLASS MaterialComponent     // { "Texture.glsl", 0xFFFFFF }, shader, color
  STATIC id = 128
// etc...
```

Therefore, quering the correct component is now simply a matter of using bitmask `OR` operator, allowing us to properly define the vectors `movements`, `transforms`, `sprites`, etc from the last example:

```
FUNCTION query(Components...)
  bitMask = 0
  FOR EACH Component IN Components
    bitMask = bitmask | Component.id

  selectedVectors = {}
  FOR EACH vector IN componentVectors
    IF vector.mask & bitMask == bitMask THEN
```

```
      selectedVectors = selectedVectors + [vector]
  RETURN selectedVectors


// ... later in the systems section:
movements, positions = query(TransformComponent.id, VelocityComponent.id)
FOR EACH movementComponent, transformComponent IN movements, positions
    // move
```

The query function now allows us to filter specific components that we want. `componentVectors` isn't defined yet but that is on purpose. For now, we also need to worry about the accessing specific members without using `query`.

## 4.3 ECS AS REMEDY: ENTITY

Why? Because not everything can be accessed and processed using systems and queries, highly specific functions like firefly's random movement and enemy's following the player still need to be processed in a OC loop. Or at least, we don't know if they will become their own systems, which means a good architecture will allow both options to the developer.

Previously, we accessed the component in a OC way by using `velocity = velocities[row]`, however, there is no guarantee that `row` will always be the correct memory address. Specificlly because we might need to change order, remove and break memory into different chunks when implementing an ECS.

One solution is to just create a second new reference table that maps all real objects into their respective rows. The underlying ECS registry will them take care of guaranteing a correct map. Each object will have its own identification, which we call "entity".

"Entity" has a really simple definition: it's an integer

| Entity | 0 | 1 | 2 | 3 | ⋯ |
|---|---|---|---|---|---|
| Row | 12 | 3 | 8 | 15 | ⋯ |

If we implement this into an map, we just need to make a two redirections to access the underlying data:

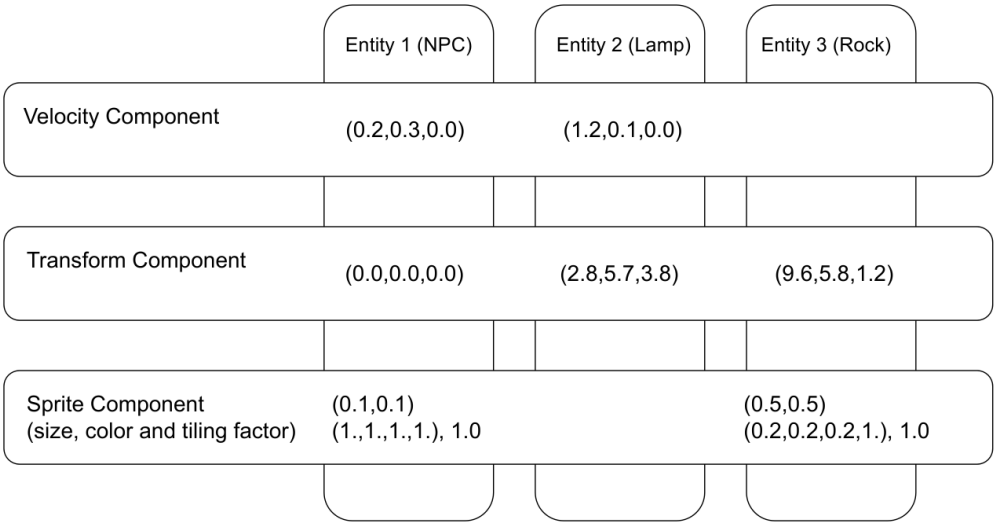
```
// NOTE: not the end function
FUNCTION get(component, entity)
  CONST row = map[entity] // first redirection
  RETURN component[row] // second redirection
```

As a benefit, we also have an immuatable, reserved identification that can be used on any object, which allow for easy debugging. While the ECS can manipulate any row it wants behind the scenes while keeping the entities intact.

There are different views for what constitutes an entity or what it's supposed to represent. For example, if you search on Wikipedia for ECS, they will define "Entity" as a general-purpose object, id est, a game object. Some articles at medium also define that way. In contrast, the Entity Systems Wiki defines as a container which components can be added. I, however, decided to go with the definition used by Unity Engine: Which states that entities are just indices which represent objects IDs.

The reason to prefer this definition it's because it's closer to the usual implementations which uses as an ID.

Figure 3 – Arbitrary components from arbitrary entities filled with arbitrary data in a random order




Source: [SOURCE TO BE CONFIRMED]

As you can guess from the definitons of components, entities and rows, the insides of ECSs is usually stored inside flexible data structures like vectors, maps, unordered maps, etc. The engine need that flexibility to maximize the contiguous memory while avoiding creating unused memory.

To achieve this, it is possible and create a separate layer, usually called "registry", which will own all the runtime data from the engine. As a didatic strategy, it can be compared to application programming interfaces (APIs).

## 4.4 ECS AS REMEDY: REGISTRY

An API is a set of rules and protocols that allows different softwares to communicate data between themselves automatically, usually into a common database but also largely used in multi services setups. In the same way the ECS registry is an "API" that allows the engine to communicate with the ECS. Except that they are almost always built in a static library once for performance reasons.

Two important routes were already shown. The first is the `query` function, which allows the engine to access contiguous memory. The second is the `get(component,entity)` function, which allows scripts like functions to occasionally access specific data from a specific entity. Both can be used as view-only or allow modifications, e.g.: C++ and rust allow const and mut logic respectivelly.

As previously stated, the registry can be responsible for:

- Retrieve the iterator, which allow us to query components in contiguous memory
- Get specific component for a given ID, which allows OC scripts to function

Going a little deeper inside the code, it can also do the following:

- Create new entity IDs and recycle destroyed IDs.
- Batch emplace new components for new IDs
- Remove IDs and their indexed components
- Check if component exist for a given ID
- Rarely, add new components to existing IDs
- Very rarely, remove components from existing IDs

Registries are hard to program because they are accessed almost every time, making them very performance critical. Usually requiring different template specific code to be able to run as clean as possible, making the developer interaction with the registry feels as if it isn't even there.

## 4.5 ECS AS REMEDY: ARCHETYPES REGISTRIES

With the registry being built we can now start to talk about Archetypes.

The problem is, we need alternatives against the use of a single matrix, but notice that when we had only `position` and `velocity`, our matrix was perfectly used. Why? Because both components were the only ones that existed and both classes, Firefly and Enemy, only used those components. However, when we added more components to the engine, that was no longer the case.

The answer seems simple then: for entities that use the same components, we can create a dedicated, smaller matrices, that doesn't have any unused memory. If we have $N$ classes in the game, and every class can use a different set of components, it means that we have a maximum of $N$ matrices.

Games however, have an extensive demand for different classes. Meaning that at some point, some of them will inevitably use the same set of components, which in turn, will make the registry use less than $N$ matrices.

For example, an environment with rocks, walls, trees and power poles. They can create this scenery with common components like: transform, material, collision and meshes. And the developer might not need to create different functions to deal with each.

| Component | $wall_0$ | $\cdots$ | $wall_{99}$ | $tree_0$ | $\cdots$ | $tree_{99}$ | $pole_0$ | $\cdots$ | $pole_{99}$ |
|---|---|---|---|---|---|---|---|---|---|
| **TransformComponent** | $(0,0)$ | $\cdots$ | $(0,0)$ | $(0,0)$ | $\cdots$ | $(0,0)$ | $(0,0)$ | $\cdots$ | $(0,0)$ |
| **MaterialComponent** | `{"Texture.glsl", 0xAA11FF}` | $\cdots$ | `{"Texture.glsl", 0xAA11FF}` | `{"Texture.glsl", 0x11AA22}` | $\cdots$ | `{"Texture.glsl", 0x11AA22}` | `{"Texture.glsl", 0x777777}` | $\cdots$ | `{"Texture.glsl", 0x777777}` |
| **ColliderComponent** | $(1,1)$ | $\cdots$ | $(1,1)$ | $(1,1)$ | $\cdots$ | $(1,1)$ | $(1,1)$ | $\cdots$ | $(1,1)$ |
| **MeshComponent** | `"wall.obj"` | $\cdots$ | `"wall.obj"` | `"tree.obj"` | $\cdots$ | `"tree.obj"` | `"pole.obj"` | $\cdots$ | `"pole.obj"` |

Now if we add a more complex object, like a moving car:

Table 1 – walls, trees and power poles

| Component | $wall_0$ | $\cdots$ | $wall_{99}$ | $tree_0$ | $\cdots$ | $tree_{99}$ | $pole_0$ | $\cdots$ | $pole_{99}$ |
|---|---|---|---|---|---|---|---|---|---|
| **TransformComponent** | $(0,0)$ | $\cdots$ | $(0,0)$ | $(0,0)$ | $\cdots$ | $(0,0)$ | $(0,0)$ | $\cdots$ | $(0,0)$ |
| **MaterialComponent** | `{"Texture.glsl", 0xAA11FF}` | $\cdots$ | `{"Texture.glsl", 0xAA11FF}` | `{"Texture.glsl", 0x11AA22}` | $\cdots$ | `{"Texture.glsl", 0x11AA22}` | `{"Texture.glsl", 0x777777}` | $\cdots$ | `{"Texture.glsl", 0x777777}` |
| **ColliderComponent** | $(1,1)$ | $\cdots$ | $(1,1)$ | $(1,1)$ | $\cdots$ | $(1,1)$ | $(1,1)$ | $\cdots$ | $(1,1)$ |
| **MeshComponent** | `"wall.obj"` | $\cdots$ | `"wall.obj"` | `"tree.obj"` | $\cdots$ | `"tree.obj"` | `"pole.obj"` | $\cdots$ | `"pole.obj"` |

Table 2 – cars

| Component | $car_0$ | $\cdots$ | $car_{99}$ |
|---|---|---|---|
| **TransformComponent** | $(0,0)$ | $\cdots$ | $(0,0)$ |
| **MaterialComponent** | `{"Texture.glsl", 0xFF2211}` | $\cdots$ | `{"Texture.glsl", 0xFF2211}` |
| **ColliderComponent** | $(1,1)$ | $\cdots$ | $(1,1)$ |
| **MeshComponent** | `"car.obj"` | $\cdots$ | `"car.obj"` |
| **VelocityComponent** | $(1,0)$ | $\cdots$ | $(1,0)$ |

And that's it, meaning objects with the same components are part of the same archetype. If we add moving rocks, those will be in the same archetype as cars.

This set of smaller and 100% filled matrix is called "archetype". It allows objects with the same set of components in the same matrix:

```
CLASS Archetype
    componentMap : MAP<ComponentType, Array>  // one SoA array per component type,
    equivalent to a matrix
    entities     : ARRAY<Entity>              // row ids, kept in sync with the
    arrays above
    removers     : ARRAY<Function(Row)>    // swap-and-pop routine for each
    component type
```

Why `removers`? ECS are performance tools, so most likely they will be implement in strong typed language. The problem is that creating an array inside componentMap will result in type erasure. Meaning the program will lose the original context it used to construct it. In c++, it happens when you use `(void *)`

This is fine for reading and modifying since the functions for that will always specify which component they want. But the same isn't true for removal functions since they will either remove a specific entity or the entire archetype, without mentioning context, e.g.:

```
registry.createComponent(componentType)
registry.getComponent(ComponentType, entity)
registry.query(ComponentType1, ComponentType2, ..., ComponentTypeN)
registry.remove(entity) // developer doesn't specify component
```

The most straightforward solution is to store the type, and therefore the memory size, of the component you want to delete, into a function that will do just that for you.

Now its just a matter of completing the ohter responsabilities with specific functions:

```
// Finds the array of a component type, creating it (and its remover) on first use
FUNCTION Archetype.createComponent(componentType)
```

```
    IF componentMap CONTAINS componentType THEN
        RETURN componentMap[componentType]

    array = NEW Array()
    componentMap[componentType] = array
    removers.APPEND(FUNCTION(row) = removeFromComponent(componentType, row))
    RETURN array
```

Insertion usually goes in the same way as any vector, after retrieving the correct vector, you append the necessary components.

```
FUNCTION Archetype.push(componentType, componentValue)
    array = createComponent(componentType)
    row = array.size // the row is the current length, taken before the append
    array.append(componentValue)
    RETURN row

// NOTE: you can make this function public, while keeping individuals push private
FUNCTION Archetype.push(entity, components)
    FOR EACH (componentType, componentValue) IN components DO
        row = pushComponent(componentType, componentValue)
    entities.append(entity)
    RETURN row
```

Similarly, you can get any individual component inside the archetype by retrieving its vector first:

```
FUNCTION Archetype.getComponent(componentType)
    ASSERT(componentMap CONTAINS componentType, "Missing component type: " +
    componentType)
    RETURN componentMap[componentType]
```

And later use it to retreive the row:

```
// Raw access to one component of one row, with the bounds check the engine needs
FUNCTION Archetype.fetchComponent(componentType, row)
    array = getComponent(componentType)
    ASSERT row < array.size
    RETURN array[row]
```

There a many ways to remove a specific compoennt, here is the strategy of swaping with the last element, then decreasing the vector size.

```
FUNCTION Archetype.removeFromComponent(componentType, row)
    array = componentMap[componentType]
    SWAP array[row], array[array.size - 1]
```

```
    array.pop()
```

This function is used here to delete an entire entity, alongside all its components. Since the row has changed, you need to update the entity-row map from section 4.3, hence the return

```
FUNCTION Archetype.remove(row)
    swappedEntity = entities[entities.size - 1]

    FOR EACH remover IN removers DO
        removeFromComponent(row)

    SWAP entities[row], entities[entities.size - 1]
    entities.pop()
    RETURN swappedEntity
```

## 4.6 EXPLAINING THE ECS METHOD: SCRIPTING

After reviewing the ECS, one might wonder how difficult it would be to replace the traditional object-oriented approach entirely with the ECS approach. After all, to implement everything using systems, developers must abstract behaviors across different entities rather than defining them individually for each object.

This is where scripting comes to help. For parts of the code that aren't critical, you can more safely use object-centric styles too.

Slower, but easier and felxible approach that abstracts many specifications of your engine. That way, game and mod developers can quickly script what they want. Without diving into the engine code, and without compiling! But you can also compile if you choose to.

Consider, for example, an npc that follows the player. Its movement could be implemented as a dedicated system, but its decision-making process might involve several conditions, timers, random choices, dialogue, and other specific rules. Implementing all these behaviors through generic systems because of a single different entity could introduce unnecessary complexity.

Instead, we can attach a script to an entity and allow it to interact with the ECS through the Registry API. This provides a middle ground between the two approaches: the engine retains the performance advantages of ECS for common operations, while developers preserve the flexibility of object-oriented programming when necessary.

How? Just create a special component that holds a single pointer and put your object specific scripts there. That pointer points to an Entity ID, which can be used to access the components later. Using the fetchComponent function.

Continuing with the example of the car earlier. Assume an arbitrary example declared

without an ECS:

```
CLASS Car
  position = (0,0)
  velocity = (0,0)
  collider = (1,1)
  mesh = NULL
  material = NULL
  break = NULL
  transmission = NULL
```

To transition to the ECS, it must plan its members to be replaced by components

```
CLASS Car
  position = (0,0)      // Transform Component
  velocity = (0,0)      // Velocity Component
  collider = (1,1)      // Collider Component
  mesh = NULL           // Mesh Component
  material = NULL       // Material Component
  break = NULL          // ?
  transmission = NULL   // ?
```

We are left with specific members `break` and `transmission`, assume there is only be very few moving cars in the game, the developer might not want to create a component just to be used in single case. For that, the engine can use a special `Script Component` with simple base class

```
CLASS Scriptable
  entity = -1

CLASS ScriptComponent
  scriptable : Scriptable *
```

The specific can then be inherit:

```
CLASS Car EXTENDS Scriptable
  break = NULL
  transmission = NULL
```

Where it can accessed by simple casting.

It's not uncommon for game engines to use runtime functions as described in the 8º chapter of the `Game Engine Architecture`: Game root loop will iterate `updateSceneElements` and `renderScene` functions. Those can be executed at different frequencies, especially if FPS is locked or if the elements needs more time to process. However, there is no rule forbiding

components to reference equivalent functions:

```
CLASS ScriptComponent
  scriptable : Scriptable *
  updateSceneElements : FUNCTION*
  renderScene : FUNCTION*

// later in the game loop
FUNCTION updateSceneElements()
  FOR EACH script IN registry.query(ScriptComponent.id)
    script.updateSceneElements()

// later in the render loop
FUNCTION renderScene()
  FOR EACH script IN registry.query(ScriptComponent.id)
    script.renderScene()
```

Because the Scriptable holds the entity ID, it can access all its original members, but as components.

Looking back to the registry, it's possible to define a new function responsable for retrieving any set of components from the entity:

```
FUNCTION Archetype.extractRow(componentTypes, row)
    row = NEW Tuple()
    FOR EACH componentType IN componentTypes DO
        row.append(fetchComponent(componentType, row))
    RETURN row
```

Since it will be used in very small cases, it might actually be faster then using the ECS system as it doesn't need to prepare the query beforehand.

This solution is often used together with a scripting language, which also abstract the C++ to a more simple and friendly language, such as Python, Lua, C#, JavaScript, etc.

# 5 RESULTS

To fully explore and validate the expected results, I design simulations that stress-test my game engine. These tests serve both to highlight its capabilities and to identify areas for optimization. Performance considerations will be emphasized, requiring scenarios where computational efficiency has a significant impact.

The expected outcomes include:

- A measurable increase in performance compared to baseline implementations.

- Clearly defined and modular systems benefiting from ECS, with increasing levels of memory layout
- A scripting system that is performant when compared to traditional OOP styles
- A robust registry system that doesn't slow down on query calls

## 5.1 TESTS

To achieve these results, I implemented and tested my engine using the following scenarios:

- **Big Entities Scene:** Design a small scene with a increasing number of particles and objects, to intentionally push the system to its limits.
- **Memory Size Test:** Design a small scene with a fixed number of particles and objects, then intentionally increase the memory size of each struct.
- **Particule Simulations:** Implement simulations involving water or numerous colliding particles, scenarios commonly found in physics-heavy games. This will validate the engine's physics system and its ability to handle numerous calculations efficiently.

Every test was built with object and archetype versions, compiled at -O2 and for 300 frames. Then retrieve a wall time of each correspondent scope.

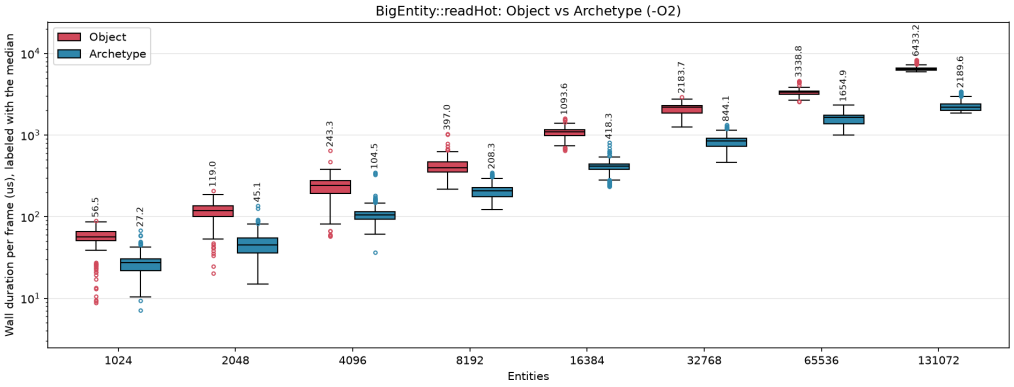
I report the median and quartiles per frame, plus the ratio Object / Archetype in a table, so a value above 1 means the Archetype version is faster. The tests ran on 11th Gen Intel i5-1135G7, 24GiB of Ram, kernal 7.2.11-arch1-1 and Clang 23.1.3. Plot scripts and raw tables are in `tcc/tests/`

## 5.2 BIG ENTITIES SCENE

A synthetic test with nothing moving or rendered. Each entity holds 1 KiB in four 256-byte components, and the count doubles from 1024 to 131072. Two read-only passes run each frame: `readHot` sums one component, and `readAll` sums all four. Both versions run the same summing loop, and the sums must match.

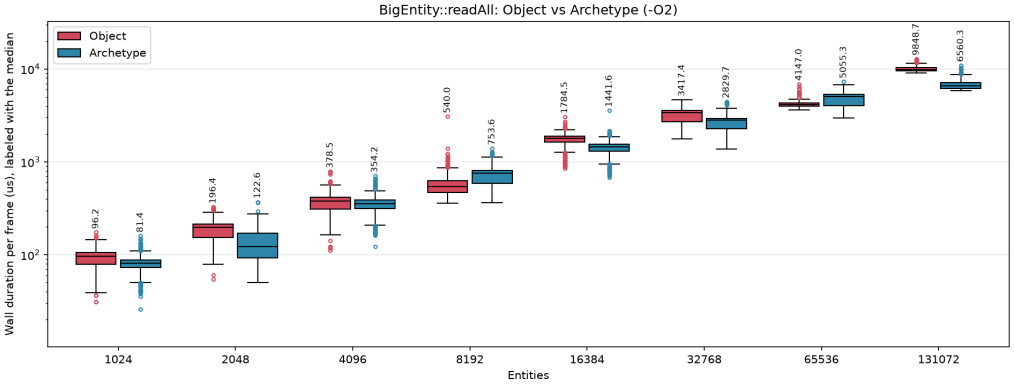
`readHot` was 2.08 to 2.80 times slower in the Object version at every entity count. `readAll` was 1.09 to 1.19 times slower, except for one outlier at 65536 entities (0.79). The cost of the layout therefore depends on how much of the entity a pass reads.

Figure 4 – Big Entities, time per frame reading one component of each entity




Source: The author.

Figure 5 – Big Entities, time per frame reading all components of each entity



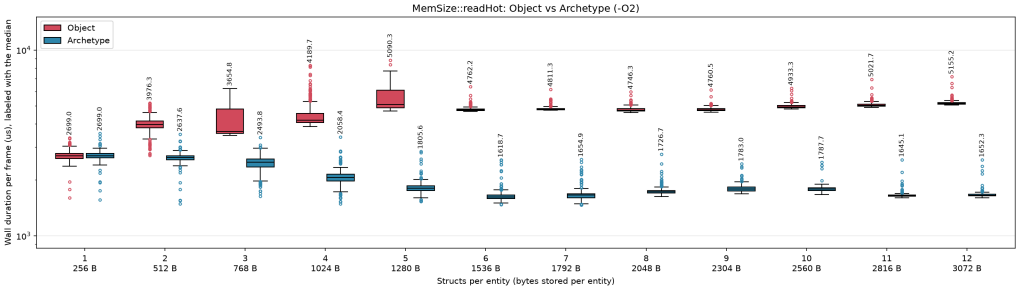

Source: The author.

## 5.3 MEMORY SIZE TEST

The entity count is fixed at 100000 and each round adds a 256-byte struct, from 1 to 12 (256 B to 3072 B per entity). `readHot` always reads one struct, so the bytes read stay constant while the bytes stored grow. `readAll` reads every struct.

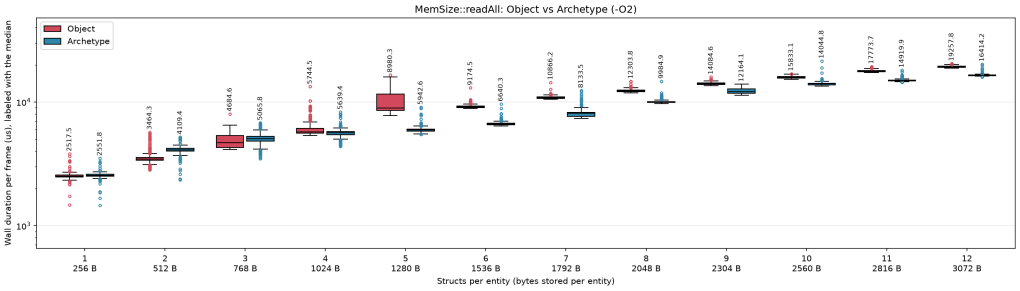
With one struct both layouts are equivalent, and `readHot` measured 2699 us in both, a useful sanity check. With 12 structs Object took 5155 us against 1652 us for Archetype (3.12x). `readAll` shows no clear advantage up to 4 structs (0.84 – 1.02) and 1.13 – 1.51x from 5 structs on.

Figure 6 – Memory Size, time per frame reading one struct of each entity




Source: The author.

Figure 7 – Memory Size, time per frame reading all structs of each entity




Source: The author.

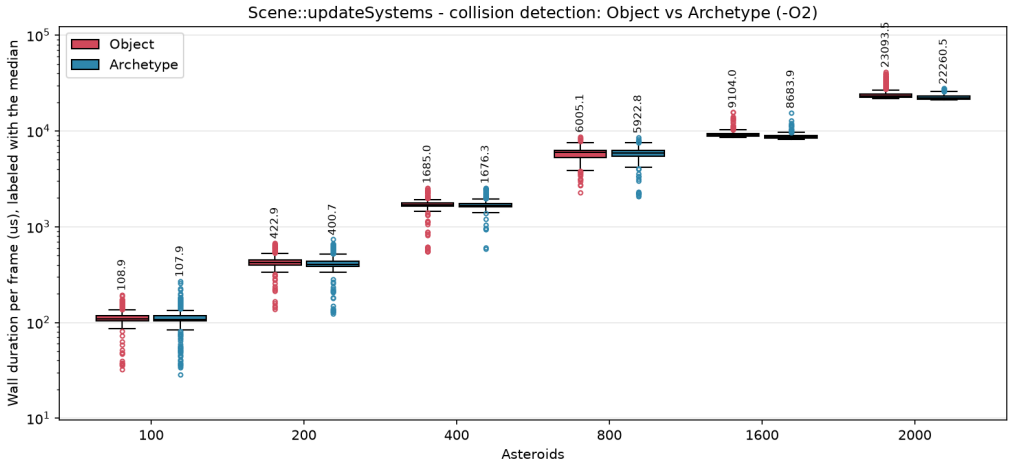
## 5.4 PARTICLE SIMULATION

A realistic scene: between 100 and 2000 asteroids move, collide with each other and with four walls, and are rendered. Initial positions and velocities come from the same seed in both versions. Collision detection is naive and O(n²).

Layout made no meaningful difference to this scene, and Object kinematics was slightly faster at low counts. Each entity is small and the systems read most of it, so the Object version wastes little cache.

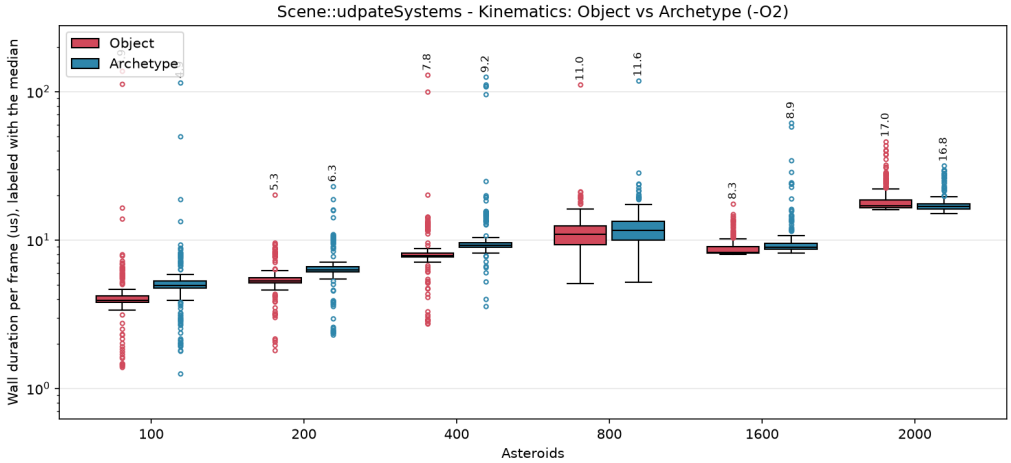
Collision also dominates the frame regardless of layout. At 2000 asteroids collision alone takes about 22 ms, way above the 16.67 ms budget of a 60 Hz frame, so the simulation no longer holds real time As stated before, `updateSceneElements` and `renderScene` runs at different frequencies, so at 22ms, render is almost never called

Figure 8 – Particle Simulation, collision detection time per frame




Source: The author.

Figure 9 – Particle Simulation, kinematics time per frame

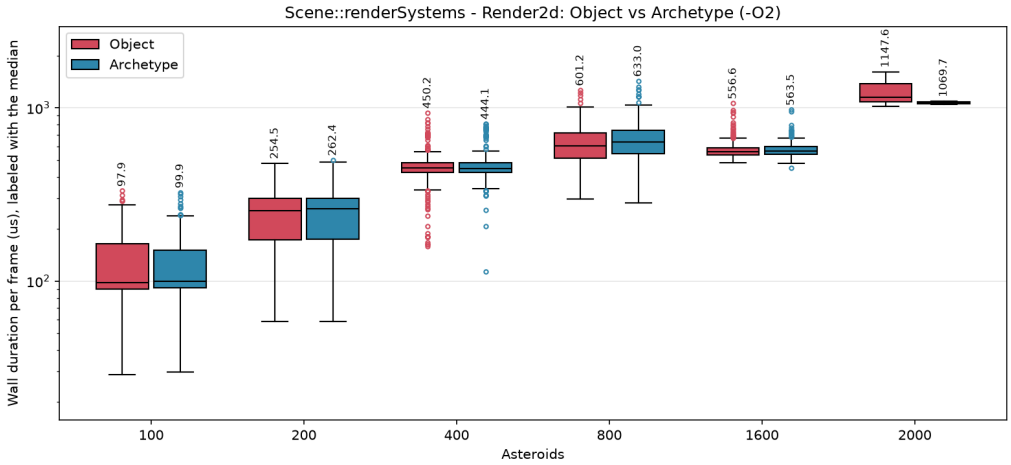



Source: The author.

## 5.5 THREATS TO VALIDITY

- Results come from one machine and 300 frames per configuration.
- The frame limit lets the CPU idle between frames, so clock changes may affect the absolute times.
- `readHot` always runs before `readAll`, so the second pass may benefit from the cache state left by the first.
- Only Object and Archetype were compared. Sparse-set and other engines remain future work.

Figure 10 – Particle Simulation, Render2d time per frame




Source: The author.

## 5.6 CONCLUSION

While game engines are never truly “finished,” the goal is to push the boundaries of what most general-purpose engines achieve within the defined scope of this project. By iterating on each of these tests and striving to exceed or match the benchmarks of existing engines, I aim to demonstrate the robustness and performance of ECS. These efforts will also provide a foundation for future comparasions, like the Sparse-Set Registry.

# REFERENCES

BRYANT, Randal E.; O'HALLARON, David R. **Computer systems: a programmer's perspective**. 2. ed. Boston: Prentice Hall, 2011.

CAINI, Michele. **ECS back and forth: Part 1 – Introduction**. skypjack on software, 14 fev. 2019. Disponível em: <https://skypjack.github.io/2019-02-14-ecs-baf-part-1/>.

CAINI, Michele. **skypjack/entt**. 2024. Disponível em: <https://github.com/skypjack/entt>. Acesso em: 23 out. 2024.

DRIESEN, Karel; HÖLZLE, Urs. The direct cost of virtual function calls in C++. **SIGPLAN Notices**, v. 31, n. 10, p. 306–323, out. 1996. DOI: 10.1145/236338.236369.

GODOT ENGINE CONTRIBUTORS. **Node class definition (node.h)**. *[S.d.]*. Disponível em: <https://github.com/godotengine/godot/blob/master/scene/main/node.h>.

LINIETSKY, Juan. **Why isn't Godot an ECS-based game engine?** Godot Engine, 26 fev. 2021. Disponível em: <https://godotengine.org/article/why-isnt-godot-ecs-based-game-engine/>.

MERTENS, Sander. **Building an ECS #1: Where are my Entities and Components**. Medium, 6 ago. 2022. Disponível em: <https://ajmmertens.medium.com/building-an-ecs-1-where-are-my-entities-and-components-63d07c7da742>.

MERTENS, Sander. **Flecs: A Fast Entity Component System (ECS) for C & C++**. *[S.d.]*. Disponível em: <https://github.com/sandermertens/flecs>.

ROMEO, Vittorio. **vittorioromeo/ecst**. 2024. Disponível em: <https://github.com/vittorioromeo/ecst>. Acesso em: 23 out. 2024.